Experiment-1

8-Puzzle problem

Aim: To implement the 8-Puzzle problem using Breadth-First Search (BFS) in Python

Algorithm:

- Start the program.
- Define the initial state and the goal state of the 8-puzzle.
- Create a queue and insert the initial state into it.
- Create a visited set to store explored states.
- Remove the front state from the queue.
- If the current state is the goal state, display the solution path and stop.
- Otherwise, generate all possible child states by moving the blank tile up, down, left, or right.
- If a child state has not been visited, add it to the queue and mark it as visited.
- Repeat Steps 5–8 until the goal state is found or the queue becomes empty.
- If the queue becomes empty, print "No solution exists."
- Stop the program.

Code:

```
from collections import deque
goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
def get_neighbors(state):
    x, y = find_blank(state)
    neighbors = []
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
```

```
            neighbors.append(tuple(tuple(row) for row in new_state))
    return neighbors
def bfs(start):
    queue = deque([(start, [])])
    visited = set()
    visited.add(start)
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [state]))
    return None
start = ((1, 2, 3),
         (4, 0, 6),
         (7, 5, 8))
solution = bfs(start)
if solution:
    print("Solution Found:\n")
    for step in solution:
        for row in step:
            print(row)
        print()
else:
    print("No solution exists.")
```

output:

```
Solution Found:

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Result

The **8-Puzzle problem** was successfully solved using the **Breadth-First Search (BFS)** algorithm in Python. BFS explored the puzzle level by level and found the **shortest path** from the initial state to the goal state.

Experiment-2

8-Queen problem

Aim: To solve the 8-Queen Problem using the Backtracking algorithm
Algorithm:

1. Create an empty 8 × 8 chessboard.

2. Start placing queens from the first column.

3. For each row in the current column:

o Check whether placing a queen is safe (no queen in the same row, upper-left diagonal, or lower-left diagonal).

4. If the position is safe:

o Place the queen.

o Recursively move to the next column.

5. If all 8 queens are placed successfully, print the solution and stop.

6. If no safe position is found in a column:

o Backtrack by removing the previously placed queen.

o Try the next possible row.

7. Repeat until a valid arrangement is found or report that no solution exists.

Code:

```
def print_board(board):
    for row in board:
        print(" ".join("Q" if x == 1 else "." for x in row))
    print()
def is_safe(board, row, col):
    # Check left side of current row
    for i in range(col):
        if board[row][i] == 1:
            return False
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
```

```
        i -= 1
        j -= 1
    i, j = row, col
    while i < 8 and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True
def solve(board, col):
    if col == 8:
        return True
    for i in range(8):
        if is_safe(board, i, col):
            board[i][col] = 1
            if solve(board, col + 1):
                return True
            board[i][col] = 0
    return False
board = [[0 for _ in range(8)] for _ in range(8)]
if solve(board, 0):
    print("Solution Found:\n")
    print_board(board)
else:
    print("No Solution Exists")
```

```
Solution Found:

Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
```

Result:

The **8-Queen problem** was successfully solved using the **Backtracking** algorithm in Python. The algorithm placed all eight queens on the chessboard such that no two queens attacked each other.

Experiment-3

Water Jug Problem

Aim:

To solve the Water Jug Problem using the Breadth-First Search (BFS) algorithm to obtain the required amount of water**.**

Algorithm:

1. Initialize two jugs with capacities 4 liters and 3 liters, both empty.
2. Represent each state as (Jug1, Jug2).
3. Create a queue and insert the initial state (0, 0).
4. Mark the visited states to avoid repetition.
5. Generate all possible next states:
6. Fill Jug1.
7. Fill Jug2.
8. Empty Jug1.
9. Empty Jug2.
10. Pour water from Jug1 to Jug2.
11. Pour water from Jug2 to Jug1.
12. Repeat the process until the target amount (2 liters) is found.
13. Print the sequence of states from the initial state to the goal state.

Code:

```
from collections import deque

jug1 = 4

jug2 = 3

target = 2

def water_jug():

    visited = set()

    queue = deque([((0, 0), [])])

    while queue:

        (x, y), path = queue.popleft()

        if (x, y) in visited:

            continue

        visited.add((x, y))

        path = path + [(x, y)]
```

```
        if x == target or y == target:
            return path
next_states = [
            (jug1, y),
            (x, jug2),
            (0, y),
            (x, 0),
            (x - min(x, jug2 - y), y + min(x, jug2 - y)),
            (x + min(y, jug1 - x), y - min(y, jug1 - x))
        ]


        for state in next_states:
            if state not in visited:
                queue.append((state, path))
    return None
solution = water_jug()
    print("Solution Path:")
    for state in solution:
        print(state)
else:
    print("No Solution Exists")
```

```
= RESTART: E:\ai\lab\water.py
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
>>
```

Result:

The Water Jug problem was successfully solved using the State Space Search technique in Python. The algorithm found a valid sequence of operations to obtain the required amount of water.

Experiment-4

Cript-Arithmetic problem

Aim:

To solve the Crypt-Arithmetic Problem using Python by assigning unique digits to letters so that the arithmetic equation is satisfied.

Algorithm:

1. Define the crypt-arithmetic equation (e.g., SEND + MORE = MONEY).
2. Extract all unique letters from the equation.
3. Generate all possible digit assignments (0–9) for these letters.
4. Ensure that no two letters have the same digit.
5. Ensure leading letters are not assigned the digit 0.
6. Replace the letters with the assigned digits.
7. Check whether the arithmetic equation is satisfied.
8. If a valid assignment is found, display the solution; otherwise, report that no solution exists.

Code:

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for p in permutations(range(10), 8):
    d = dict(zip(letters, p))
    if d['S'] == 0 or d['M'] == 0:
        continue
    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    MONEY = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']
    if SEND + MORE == MONEY:
        print("Solution Found:")
        print("SEND =", SEND)
        print("MORE =", MORE)
        print("MONEY =", MONEY)
        print("\nLetter Assignments:")
        for k, v in d.items():
            print(k, "=", v)
        break
```

```
---------------------------- RESTART: E:\ai\lab\cp.py -------------
Solution Found:
SEND = 9567
MORE = 1085
MONEY = 10652

Letter Assignments:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
```

Result:

The **Crypt-Arithmetic problem** was successfully solved using the **Constraint Satisfaction (Backtracking)** approach in Python. The algorithm assigned unique digits to letters while satisfying the given arithmetic equation.

Experiment-5

Missionaries and Cannibals Problem

Aim:

To solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm in Python.

Algorithm:

1. Represent each state as (Missionaries Left, Cannibals Left, Boat Side).
2. Start with the initial state (3, 3, 1) where all missionaries, cannibals, and the boat are on the left bank.
3. Use Breadth-First Search (BFS) to explore all valid states.
4. Generate the next possible states by moving:
5. 1 Missionary
6. 2 Missionaries
7. 1 Cannibal
8. 2 Cannibals
9. 1 Missionary and 1 Cannibal
10. Discard invalid states where cannibals outnumber missionaries on either bank.
11. Continue until the goal state (0, 0, 0) is reached.
12. Print the sequence of states from the initial state to the goal state.

Code:

```
from collections import deque

def valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and c > m:
        return False
    mr = 3 - m
    cr = 3 - c
    if mr > 0 and cr > mr:
        return False
    return True

def bfs():
    start = (3, 3, 1)   # (Missionaries, Cannibals, Boat)
    goal = (0, 0, 0)
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
```

```
    queue = deque([(start, [start])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        if state in visited:
            continue
        visited.add(state)
        m, c, boat = state
        for dm, dc in moves:
            if boat == 1:   # Boat on left bank
                new = (m-dm, c-dc, 0)
            else:          # Boat on right bank
                new = (m+dm, c+dc, 1)
            if valid(new[0], new[1]) and new not in visited:
                queue.append((new, path + [new]))
    return None
solution = bfs()
if solution:
    print("Solution Path:")
    for state in solution:
        print(state)
else:
    print("No Solution Exists")
```

Output:

```
= RESTART: E:\ai\lab\cmp.py
Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

Result:

The Missionaries and Cannibals problem was successfully solved using the State Space Search algorithm in Python. The algorithm generated a valid sequence of moves that safely transported all missionaries and cannibals across the river without violating the problem constraints.

Experiment-6

Vacuum Cleaner Problem

Aim

To implement the **Vacuum Cleaner Problem** in Python, where a vacuum agent cleans all dirty

Algorithm

1. Initialize the environment with two rooms (A and B) and their states (Clean/Dirty).
2. Place the vacuum cleaner in one of the rooms.
3. Check the current room:
4. If it is Dirty, clean it.
5. If it is Clean, move to the other room.
6. Repeat the process until both rooms are clean.
7. Display each action performed by the vacuum cleaner.
8. Stop when all rooms are clean.

Code:

```
rooms = {'A': 'Clean', 'B': 'Dirty'}

location = 'A'


while True:
    if rooms[location] == 'Dirty':
        print("Cleaning Room", location)
        rooms[location] = 'Clean'
    else:
        print("Room", location, "is already Clean")
    if rooms['A'] == 'Clean' and rooms['B'] == 'Clean':
        print("\nAll Rooms are Clean")
        break
    if location == 'A':
        location = 'B'
    else:
        location = 'A'
print("\nFinal Room Status:")
for room in rooms:
    print(room, ":", rooms[room])
```

output:

```
---------- RESTART: E:\ai\lab\vacuum.py -----
Room A is already Clean
Cleaning Room B

All Rooms are Clean

Final Room Status:
A : Clean
B : Clean
```

Result:

The Vacuum Cleaner problem was successfully solved using a Simple Reflex Agent in Python. The agent sensed the environment, cleaned all dirty rooms, and successfully reached the goal state, where all rooms were clean.

Experiment-7

Breadth First Search

Aim:

To write a Python program to implement the **Breadth First Search (BFS)** algorithm

Algorithm:

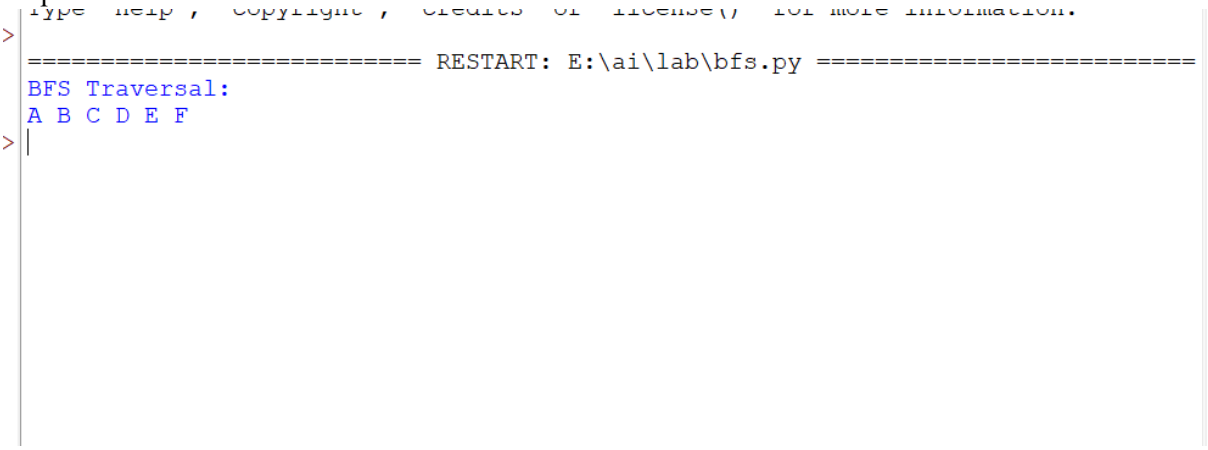
1. Start.
2. Create a graph using an adjacency list.
3. Select the starting vertex.
4. Mark the starting vertex as visited and insert it into a queue.
5. While the queue is not empty:
    - Remove the front vertex from the queue.
    - Display the vertex.
    - Visit all its unvisited adjacent vertices.
    - Mark each adjacent vertex as visited and add it to the queue.
6. Stop when all reachable vertices are visited.
7. End.

Code:

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    print("BFS Traversal:")

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
```

```
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
graph = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F'],
  'D': ['B'],
  'E': ['B', 'F'],
  'F': ['C', 'E']
}
start_node = 'A'
bfs(graph, start_node)
```

output:



Result:

Thus, the Python program to implement the **Breadth First Search (BFS)** algorithm was written and executed successfully.

Experiment-8

Depth First Search

Aim:

To write a Python program to implement the **Depth First Search (DFS)** algorithm for graph traversal.

Algorithm:

1. Start.
2. Create a graph using an adjacency list.
3. Select the starting vertex.
4. Mark the starting vertex as visited.
5. Display the current vertex.
6. Recursively visit each unvisited adjacent vertex.
7. Repeat until all reachable vertices are visited.
8. End.

Code:

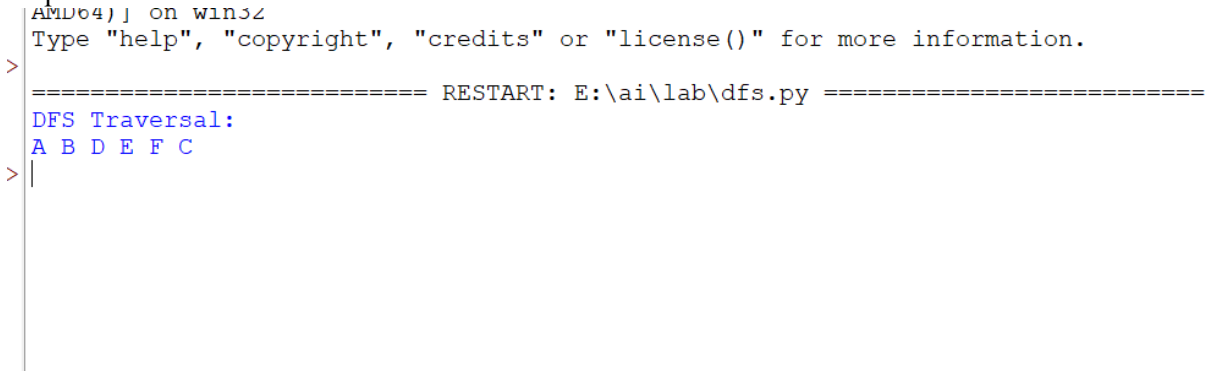
```
def dfs(graph, vertex, visited):
    visited.add(vertex)
    print(vertex, end=" ")

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

start_node = 'A'
```

visited = set()

print("DFS Traversal:")

dfs(graph, start_node, visited)

output:



Result

Thus, the Python program to implement the **Depth First Search (DFS)** algorithm was written and executed successfully.

Experiment-9

Travelling Salesman Problem

Aim:

To write a Python program to implement the **Travelling Salesman Problem (TSP)** using a brute-force approach.

Algorithm:

- Start.
- Represent the cities and distances using a distance matrix.
- Choose a starting city.
- Generate all possible permutations of the remaining cities.
- Calculate the total travel cost for each possible route.
- Select the route with the minimum travel cost.
- Display the minimum cost and the optimal path.
- End.

Code:

```
from itertools import permutations
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
n = len(graph)
start = 0
min_cost = float('inf')
best_path = []
cities = list(range(n))
cities.remove(start)

for perm in permutations(cities):
    current_cost = 0
    current_path = [start] + list(perm) + [start]
```

```
    for i in range(len(current_path) - 1):
        current_cost += graph[current_path[i]][current_path[i + 1]]

    if current_cost < min_cost:
        min_cost = current_cost
        best_path = current_path

print("Minimum Cost:", min_cost)
print("Optimal Path:", best_path)
```

output:

```
Type "help", "copyright", "credits" or "license()" for more information.

========== RESTART: E:\ai\lab\tsm.py ==========
Minimum Cost: 80
Optimal Path: [0, 1, 3, 2, 0]
```

Result:

Thus, the Python program to implement the Travelling Salesman Problem (TSP) using the brute-force method was written and executed successfully.

Experiment-10

A* algorithm

Aim

To write a Python program to implement the **A*** algorithm for finding the shortest path between two nodes in a graph.

Algorithm

1. Start.
2. Create a graph with nodes and edge costs.
3. Define a heuristic value for each node.
4. Initialize the open list with the start node and an empty closed list.
5. Select the node with the lowest **f(n) = g(n) + h(n)**.
6. If the selected node is the goal node, display the path and stop.
7. Otherwise, expand the node and update the cost of its neighbors.
8. Repeat Steps 5–7 until the goal is reached or no path exists.
9. End.

Code:

```
import heapq

def astar(graph, heuristic, start, goal):
    open_list = [(0, start)]
    came_from = {}
    g_cost = {start: 0}

    while open_list:
        current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
```

```
            path.reverse()
            return path, g_cost[goal]

        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                f_cost = new_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, float('inf')
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 6)],
    'E': [('G', 2)],
    'F': [('G', 1)],
    'G': []
}

# Heuristic values
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 4,
    'E': 2,
    'F': 1,
    'G': 0
```

```
}

start = 'A'
goal = 'G'
path, cost = astar(graph, heuristic, start, goal)
print("Shortest Path:", path)
print("Total Cost:", cost)
```

output:

```
Type "help", "copyright", "credits" or "license()" for more information.
>
= RESTART: E:\ai\lab\Astar.py
Shortest Path: ['A', 'B', 'E', 'G']
Total Cost: 8
>
```

Result:

Thus, the Python program to implement the A* (A-Star) algorithm for finding the shortest path was written and executed successfully.

Experiment-11

Map Coloring Problem

Aim

To write a Python program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.

Algorithm

1. Start.
2. Define the map as a graph where each region is a node and adjacent regions are connected.
3. Define the available colors.
4. Assign a color to each region.
5. Check that no two adjacent regions have the same color.
6. If a conflict occurs, try another color (backtracking).
7. Repeat until all regions are colored successfully.
8. Display the color assigned to each region.
9. End.

Code:

```
def is_safe(region, color, assignment, graph):
    for neighbor in graph[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True
def map_coloring(graph, colors, assignment, regions, index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_safe(region, color, assignment, graph):
            assignment[region] = color
            if map_coloring(graph, colors, assignment, regions, index + 1):
                return True
            del assignment[region]
```

```
    return False
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
assignment = {}
regions = list(graph.keys())
if map_coloring(graph, colors, assignment, regions, 0):
    print("Color Assignment:")
    for region in assignment:
        print(region, "->", assignment[region])
else:
    print("No solution exists.")
```

output:

Result:

Thus, the Python program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) with backtracking was written and executed successfully.

Experiment-12

Tic-Tac-Toe

Aim:

To write a Python program to implement the Tic-Tac-Toe game for two players.

Algorithm:

1. Start.
2. Create a 3×3 game board.
3. Display the empty board.
4. Allow two players (X and O) to take turns entering their positions.
5. Update the board after each valid move.
6. Check for a winner after every move (row, column, or diagonal).
7. If all cells are filled and there is no winner, declare the game a draw.
8. Display the result and end the game.
9. End.

Code:

```
board = [' ' for _ in range(9)]
def print_board():
    print()
    for i in range(3):
        print(board[i*3], "|", board[i*3+1], "|", board[i*3+2])
        if i < 2:
            print("--+---+--")
    print()
def check_winner(player):
    win_positions = [
        [0,1,2], [3,4,5], [6,7,8],   # Rows
        [0,3,6], [1,4,7], [2,5,8],   # Columns
        [0,4,8], [2,4,6]             # Diagonals
    ]
```

```
    for pos in win_positions:
        if all(board[i] == player for i in pos):
            return True
    return False
def is_draw():
    return ' ' not in board
player = 'X'
while True:
    print_board()
    try:
        move = int(input(f"Player {player}, enter position (1-9): ")) - 1
    except ValueError:
        print("Enter a valid number.")
        continue
    if move < 0 or move > 8:
        print("Position must be between 1 and 9.")
        continue
    if board[move] != ' ':
        print("Position already occupied.")
        continue
    board[move] = player
    if check_winner(player):
        print_board()
        print(f"Player {player} Wins!")
        break
    if is_draw():
        print_board()
        print("Game Draw!")
        break
    player = 'O' if player == 'X' else 'X'
```

output:

```
   | X |
--+---+--
   |   |
--+---+--
   |   |

Player O, enter position (1-9): 1

O | X |
--+---+--
   |   |
--+---+--
   |   |

Player X, enter position (1-9): 5

O | X |
--+---+--
   | X |
--+---+--
   |   |

Player O, enter position (1-9): 3

O | X | O
--+---+--
   | X |
--+---+--
   |   |

Player X, enter position (1-9): 8

O | X | O
--+---+--
   | X |
--+---+--
   | X |

Player X Wins!
```

Result

Thus, the Python program to implement the Tic-Tac-Toe game for two players was written and executed successfully.

Experiment-13

Minimax Algorithm

Aim:

To write a Python program to implement the Minimax Algorithm for decision-making in a game.

Algorithm:

1. Start.
2. Represent the game tree with terminal node values.
3. Define a recursive Minimax function.
4. If the current node is a terminal node, return its value.
5. If it is the maximizing player's turn, return the maximum value of its children.
6. If it is the minimizing player's turn, return the minimum value of its children.
7. Continue recursively until the optimal move is found.
8. Display the optimal value.
9. End.

Code:

```
import math
def minimax(depth, node_index, is_max, values, max_depth):
    # Terminal node
    if depth == max_depth:
        return values[node_index]

    if is_max:
        return max(
            minimax(depth + 1, node_index * 2, False, values, max_depth),
            minimax(depth + 1, node_index * 2 + 1, False, values, max_depth)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, values, max_depth),
```

```
            minimax(depth + 1, node_index * 2 + 1, True, values, max_depth)
        )
values = [3, 5, 2, 9, 12, 5, 23, 23]
max_depth = int(math.log2(len(values)))
result = minimax(0, 0, True, values, max_depth)
print("Optimal Value:", result)
```

output:

```
========================= RESTART: E:\ai\lab\minimax.py =========================
Optimal Value: 12
```

Result:

Thus, the Python program to implement the Minimax Algorithm for gaming was written and executed successfully.

Experiment-14

Alpha-Beta Pruning Algorithm

Aim:

To write a Python program to implement the **Alpha-Beta Pruning Algorithm** for optimizing the **Minimax** search in game playing.

Algorithm

1. Start.
2. Represent the game tree with terminal node values.
3. Initialize **alpha** = -∞ and **beta** = +∞.
4. If the current node is a terminal node, return its value.
5. If it is the maximizing player's turn:
    - Find the maximum value among its children.
    - Update **alpha**.
    - Prune the remaining branches if **alpha ≥ beta**.
6. If it is the minimizing player's turn:
    - Find the minimum value among its children.
    - Update **beta**.
    - Prune the remaining branches if **beta ≤ alpha**.
7. Continue recursively until the optimal value is found.
8. Display the optimal value.
9. End.

Code:

```
import math
def alpha_beta(depth, node_index, is_max, values, alpha, beta, max_depth):
    # Terminal node
    if depth == max_depth:
        return values[node_index]
    if is_max:
        best = -math.inf
        for i in range(2):
            value = alpha_beta(depth + 1,
```

```
                    node_index * 2 + i,
                    False,
                    values,
                    alpha,
                    beta,
                    max_depth)
            best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            value = alpha_beta(depth + 1,
                    node_index * 2 + i,
                    True,
                    values,
                    alpha,
                    beta,
                    max_depth)
            best = min(best, value)
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best
values = [3, 5, 6, 9, 1, 2, 0, -1]
max_depth = int(math.log2(len(values)))
result = alpha_beta(0, 0, True, values, -math.inf, math.inf, max_depth)
print("Optimal Value:", result)
```

output:

```
========== RESTART: E:\ai\lab\alphabeta.py ==========
Optimal Value: 5
>
```

Result

Thus, the Python program to implement the Alpha-Beta Pruning Algorithm for gaming was written and executed successfully.

Experiment-15

Decision Tree

Aim

To write a Python program to implement a Decision Tree Classifier using the scikit-learn library.

Algorithm

1. Start.
2. Import the required libraries.
3. Load a dataset.
4. Split the dataset into training and testing sets.
5. Create a Decision Tree classifier.
6. Train the classifier using the training data.
7. Predict the class labels for the test data.
8. Calculate and display the accuracy of the model.
9. End.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Labels:", y_pred)
print("Accuracy:", accuracy)
```

Output:

```
= RESTART: E:\ai\lab\dtree.py
Predicted Labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0
 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
Accuracy: 1.0
>
```

Result

Thus, the Python program to implement the Decision Tree Classifier was written and executed successfully.

Experiment-16

Feed Forward Neural Network

Aim

To write a Python program to implement a Feed Forward Neural Network (FFNN) using the scikit-learn library.

Algorithm

1. Start.
2. Import the required libraries.
3. Load the Iris dataset.
4. Split the dataset into training and testing sets.
5. Create a Feed Forward Neural Network model using MLPClassifier.
6. Train the model using the training data.
7. Predict the class labels for the test data.
8. Calculate and display the accuracy of the model.
9. End.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=42
```

)

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Labels:", y_pred)
print("Accuracy:", accuracy)
```

output:

```
Predicted Labels: [1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0
 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
Accuracy: 0.9777777777777777
>
```

Result

Thus, the Python program to implement a Feed Forward Neural Network (FFNN) using the MLPClassifier was written and executed successfully.